st_03_paradigms

1. Paradigms

In the ancient ages (up to the medieval age) natural philosophers believed that all of the physical world is composed out of four elements, namely earth, water, air, and fire. Additionally, these four elements were used to explain illness and to design medical treatments (humoral pathology), and they served as explanations in psychology.
Is the theory of four elements a paradigm? If so, is it fine grained or coarse-grained? Please, justify!

2. Imperative Paradigm, Examples (1)

Check out Basic-program quad_eq.bas from repository. Enter the URL https://www.jdoodle.com/execute-freebasic-online in your web browser. Insert the Basic-program in the browser-shell for Basic. Be careful to enable the interactive mode. Execute the program and enter different values. Try to find suitable input values so that different runs of the programs yield each possible output (“No quadratic equation” … two solutions).

3. Imperative Paradigm, Examples (1)

Check out Basic-program quad_eq.bas from repository. Remove the blank lines and apply line numbering to the lines. Tell the ranges of line numbers which you may change in any order without breaking the program or affecting it visible behavior, tell the ranges of line numbers which you may reorder only a little, and tell the line ranges where you may not apply any change.

4.Structured Paradigm

Structured Programming banned GOTO because of the harmful consequences of using it. Early Basic-dialects offered GOSUB and RETURN. GOSUB allowed to jump to a line number and RETURN returned to the instruction (respectively line number) immediately following the GOSUB-statement. It was not possibly to pass any parameters using GOSUB.
Does GOSUB-RETURN avoid the negative properties of GOTO? Which deficits do still apply?

5. Object-oriented Paradigm, Example in C++ (2)

This exercise has to be done using C++ exclusively. But you will be provided with additional information to master the challenge. Nevertheless, some important things are left to you. Carefully study the slide “Example in Modula-2 (2)”. Be sure to exactly understand the code provided here. Now open http://cpp.sh in your web browser.
In the following you will have to develop several functionally equivalent versions of a C++ program printing random numbers, each according to one of the various imperative paradigms. First, the program, running in the console, should ask, whether the (pseudo) random number generator should be randomly seeded. Legal answers are y for yes or any other character for no. Then a (pseudo) random number is printed and the program asks, whether another random number should be printed. Legal answers are y for yes or any other character for no. Here is an example dialog:

```
Random seed the generator (y/n)? y
266
Another random number (y/n)? y
243
Another random number (y/n)? y
914
Another random number (y/n)? y
947
Another random number (y/n)? n
Program ended with exit code: 0
```

Well, now for the various versions of the program you should develop.

1. The first version is purely imperative with a minimal set of required statements, namely an if with a single branch only, and X (something you have to find out). Using any other branching statement, looping statement, or functions (with the exception of function main()), or recursion is strictly prohibited. You can checkout random_minimal_frame.cpp from the repository as a template. It contains additional hints for the implementation in C++.
2. The second version should comply exactly with structured programming, i.e., you may use any branch-statement, any loop-statement, functions, but **not** ingredient X. Furthermore, you are not allowed to define structs or classes. Use your implementation of the first version as a starting point. Try to define as many functions as useful in order to improve the structure and the readability of your program. In case you should get an error with respect to using max, simply write ::max. Now the error should vanish, since max will be search in the top-level space of your program and not in namespace std;
3. The third version should use an abstract data object for encapsulating the (pseudo) random number generator. You are not allowed to use inheritance and you may not create instances of your abstract data object. The data of the ADO may not be publicly accessible, instead access functions have to be used. The main challenge is to find the appropriate implementation for an ADO in C++ (btw., it would be the same in Java). Furthermore, you should avoid any global variable. Maybe you need to initialize static data member of a class. This has to be done outside the class, while the declaration has – of course – to be inside the class.
4. For implementing the fourth version you should use an ADT. This time you will have to implement a functional change to the program. It now has to include two (pseudo) random number generators, which can be randomly seeded individually. Consequently, in each iteration of the program, two (pseudo) random numbers are to be printed.

6. Sequence Does not Matter, BUT …

Carefully study slide “Expert System”. Check out the XPS for risk assessment. Start the XPS shell ESIE in a DOSBox, load RISK.TXT and execute it. Now start swapping the lines in RISK.TXT, either systematically or more or less randomly. Save the text file, reload it from ESIE and start it again.

a) Provided you enter the same data, does the dialog change because of line swaps?
b) How many versions of RISK.TXT with swapped lines (not counting blank lines), that are semantically equivalent, do exist?

Be careful not to change the name RISK.TXT. DOSBox is neither case sensitive, nor does it process filenames longer than 8 + 3. When editing RISK.TXT be sure to use an editor that does not introduce extra characters or uses a character code different from ASCII.

Important Notice if you want to try it out.

If you try to do this exercise at home, please read the following carefully. Remember you could use your Virtual Machines.

1) Download the ZIP archive ESIE.zip from our Moodle course and unzip it to a directory of your choice.
2) Search for the DOS-emulator DOSBox and install it.
    a. If you are using Linux open a terminal and switch to the directory containing ESIE. Type “dosbox ESIE” and press ENTER. DOSBox should start now and mount the ESIE directory automatically. Start ESIE by typing “esie” in DOSBox.
    b. If you are using the Windows virtual machine or downloaded the emulator for Windows, then simply drag and drop the ESIE.COM on the DOSBox Icon.
    c. If you are using MacOS change to directory /Users/YOUR_USER_DIR/Library/Preferences, where YOUR_USER_DIR equals the home directory of your user. After switching to this directory type “vi DOS*”- Scroll to the end of the file and press “a” for “append”. Append the following two lines (replace USER_DIR by the name of the home directory of your user and ESIE_DIR by the directory to which you unzipped ESIE.ZIP):
    keyb de453
    mount c /Users/USER_DIR/ESIE_DIR
    Press ESC-key and then “ZZ”. “Esc” leaves input mode, “ZZ” writes the file to disk and quits VI. The first line “keyb de453” is required to start DOSBox with a German keyboard layout.
    Afterwards you can start DOSBox from the launcher.

7. Imperative vs. Logic/Declarative Programming

Compare the following two programs in Prolog and C++ and how these relate to the corresponding paradigms.

Prolog

```
even(X) :- X mod 2 =:= 0.
first_even([H|_], H) :- even(H), !.
first_even([_|T], X) :- first_even(T, X).

% Example query
?- first_even([3, 7, 8, 5, 10], X).
```

C++

```
#include <iostream>
#include <vector>
using namespace std;

int firstEven(const vector<int>& numbers) {
   for (int num : numbers) {
      if (num % 2 ==0) {
         return num;
```

```
        }
    }
    return -1;
}

int main() {
    vector<int> numbers = {3, 7, 8, 5, 10};
    cout <<  firstEven(numbers) << endl;

    return 0;
}
```

8. Artificial Neural Networks, Benefits and Drawbacks (at Present Time)

Carefully study the slides on “Software 2.0”. Now look back at the four program versions you created in assignment “Object-oriented Paradigm, Example in C++ (2)”. Suppose, you have to train a neural network. Which of the four versions would you use for training and why?

9. Paradigms

Please inform yourself about the eight queens problem.
https://en.wikipedia.org/wiki/Eight_queens_puzzle Which paradigm and or programming language would you choose to to tackle the problem and why?

10. SPL

Provided to you is the file matrix_2x2_double.h.
You get tasked with creating a product line for this header, so that the functions can be put in or taken out on demand and you can change the datatype of the matrix.
How would tackle such a problem in C++?
What problems arise when each function can be switched on or off and the datatype is exchangeable?
Think about how you could automate the generation of the different products of this product line, what would be your approach?